

Claude Code prin exemple

Toate piesele atelierului, fiecare cu un exemplu care rulează pe cele 5 repo-uri și pe clusterul tău

Cum se citește: fiecare piesă are patru lucruri și doar patru — **ce e** în două rânduri, **exemplul complet** (fișierul, așa cum îl scrii), **poarta** (cum verifici că piesa chiar face ce zice) și **când NU** o folosești. Nimic nu e teorie: toate exemplele sunt scrise pe `data-service`, `importer-service`, `client-microserv-vite`, `argo-ms-gitops` și `ms-gitops`.

Regula pe care stă tot atelierul: agentul nu atinge clusterul. Agentul schimbă repo-ul. Clusterul se schimbă singur.

Nu e o regulă despre AI, e GitOps-ul pe care îl faci deja. Diferența e că un agent poate face zece `kubectl apply` în treizeci de secunde, fără să obosească și fără să ezite.

Harta — cine pornește fiecare piesă

Asta e singura diagramă din ghid și e singura pe care merită s-o ții minte. Toate cele cinci piese fac același lucru — pun context sau instrucțiuni în fața agentului. Se deosebesc **prin cine apasă pe buton**:

Piesa	Cine o pornește	Unde stă fișierul	La ce o folosești
CLAUDE.md	nimeni — se citește la fiecare pornire	<code>ms/CLAUDE.md</code>	ce ai repeta în orice conversație
permisiuni	nimeni — sunt verificate la fiecare comandă	<code>ms/.claude/settings.json</code>	ce n-are voie să atingă, niciodată
comanda /nume	tu , pe nume	<code>ms/.claude/commands/nume.md</code>	o cerere pe care o repeți identic
skill-ul	agentul , când se potrivește situația	<code>ms/.claude/skills/nume/SKILL.md</code>	o procedură pe care o uiți tu
subagentul	tu sau agentul, ca delegare	<code>ms/.claude/agents/nume.md</code>	o treabă mare, cu alt rol și alte drepturi
hook-ul	nimeni — se declanșează pe eveniment	<code>ms/.claude/settings.json</code>	ce trebuie să se întâmple chiar dacă uiți

Confuzia clasică, rezolvată o dată: comanda și skill-ul conțin același lucru — un text cu instrucțiuni. Diferența nu e în conținut, e în declanșator. Comanda o chemi tu, scriind `/lant`. Skill-ul îl cheamă agentul singur, citind linia `description` din el și hotărând că se potrivește cu ce i-ai cerut. Dacă vrei control, scrii comandă. Dacă vrei ca procedura să se aplice **și când tu ai uitat de ea**, scrii skill.

Partea 0 — Primele treizeci de minute

Pornirea și folderul — prima decizie, și cea mai importantă

0.1

Agentul vede folderul din care l-ai pornit, cu tot ce e sub el. Deci pornirea în `ms/` nu e o preferință, e diferența dintre a putea și a nu putea răspunde:

```
cd ~/ms
claude
```

Întrebarea de probă, care **taie prin două repo-uri deodată** — exact tipul de întrebare la care azi răspunzi cu trei tab-uri de GitHub:

Ce tag de imagine citește ArgoCD pentru importer-service și ce tag a scris ultima rulare de CI? Spune-mi și din ce fișiere ai citit.

Poarta: pornește-l a doua oară din `~/ms/importer-service` și pune aceeași întrebare. Nu poate răspunde — nu vede repo-ul de GitOps. Țasta e tot ce trebuie să știi despre „unde pornesc”.

Cele opt taste și comenzi care îți trebuie de la început

0.2

Ce apeși / scrii	Ce face	Când îl folosești la tine
<code>Esc</code>	oprește agentul în mijlocul lucrului	a pornit pe un drum greșit — îl oprești, nu aștepti
<code>Esc</code> de două ori	te duce înapoi la un mesaj anterior	vrei să reformulezi cererea, nu s-o corectezi din mers
<code>Shift+Tab</code>	comută modul: normal / plan / auto-accept	modul plan, înainte de orice atingere de fișiere
<code>@</code>	completează o cale de fișier în cerere	<code>@argo-ms-gitops/argo-apps/app-data.yaml</code>
<code>/clear</code>	golește conversația, păstrează <code>CLAUDE.md</code>	treci la alt subiect — nu duce gunoiul după tine
<code>/context</code>	arată cât din context e ocupat și cu ce	când răspunsurile încep să rateze detalii
<code>claude -c</code>	reia ultima conversație din folderul curent	a doua zi, pe aceeași problemă
<code>/help</code>	lista completă de comenzi	când nu-ți amintești numele exact

Modul plan (Shift+Tab) e cel pe care îl vei folosi cel mai des: agentul cercetează și îți scrie planul, dar **nu modifică nimic** până nu accepți. La o platformă cu cinci repo-uri, planul citit înainte îți arată dacă a înțeles corect care repo e care — fix greșeala care costă cel mai mult.

Chat sau agent — două întrebări care sună la fel

0.3

Ce versiune de Java se folosește în general la Spring Boot 3?

Ce versiune de Java folosește data-service-ul meu, și de unde ai aflat?

Poarta: la prima a răspuns din ce știe. La a doua **trebuie** să-ți spună fișierele — `pom.xml`, `Dockerfile`, sau amândouă. Dacă cele două fișiere nu spun același lucru, tocmai ai găsit un bug fără să-l cauți.

Regula de citire a oricărui răspuns, de aici încolo: „de unde ai aflat” face parte din întrebare. Un răspuns fără sursă e o bănuială bine scrisă.

Piesa 1 — Permisunile: ce n-are voie, niciodată

Ce e: un fișier de reguli verificat înainte de fiecare comandă. Se pune **primul**, înainte de orice unealtă — nu după prima pățanie.

ms/.claude/settings.json

```
{
  "permissions": {
    "deny": [
      "Bash(kubectl apply:*)",
      "Bash(kubectl delete:*)",
      "Bash(kubectl edit:*)",
      "Bash(kubectl scale:*)",
      "Bash(kubectl patch:*)",
      "Bash(kubectl rollout:*)",
      "Bash(helm install:*)",
      "Bash(helm upgrade:*)",
      "Bash(argocd app sync:*)",
      "Bash(argocd app set:*)",
      "Bash(git commit:*)",
      "Bash(git push:*)"
    ],
    "allow": [
      "Bash(kubectl get:*)",
      "Bash(kubectl describe:*)",
      "Bash(kubectl logs:*)",
      "Bash(helm template:*)",
      "Bash(argocd app get:*)",
      "Bash(git status:*)",
      "Bash(git log:*)",
      "Bash(git diff:*)"
    ]
  }
}
```

Cele trei liste, în ordinea în care contează: `deny` — refuză, fără să te întrebe. `allow` — rulează fără să te întrebe. Ce nu e în niciuna — **te întreabă**, și asta e comportamentul implicit, deci lista `allow` e doar pentru liniștea ta, nu pentru siguranță. Siguranța e în `deny`.

Poarta — treizeci de secunde, și nu le sări

Cere-i, exact așa:

```
Da scale la deployment-ul importer-service in business la 2 replici.
```

Trebuie să refuze, și trebuie să vezi motivul pe ecran. Apoi, imediat, opusul:

```
Cate replici are acum importer-service in business si de cate ori  
a repornit?
```

Gata când: primul refuzat, al doilea răspuns. Ai obținut poziția din care vrei să lucrezi: **un agent care vede tot clusterul și nu poate schimba nimic în el.**

Unde se oprește gardul — spune-o cu voce tare acum, nu când se întâmplă: regulile se potrivesc pe forma comenzii. Un `kubectl apply` chemat dintr-un script pe care agentul tocmai l-a scris nu arată ca `Bash(kubectl apply:*)` și nu se declanșează. Gardul e o plasă, nu un zid. Zidul rămâi tu, care citești ce rulează.

De verificat la tine: rulează `/permissions` ca să vezi regulile active și de unde vin. Fișierul din `ms/.claude/settings.json` se combină cu cel personal din `~/.claude/settings.json`, iar când ceva nu

se declanșează, de obicei e o regulă scrisă în alt fișier decât crezi.

Piesa 2 — `CLAUDE.md`: contextul care nu se mai repetă

Ce e: un fișier citit automat la fiecare pornire, din folderul în care pornești. Aici scrii **o singură dată** ce altfel ai explica în fiecare conversație.

```
ms/CLAUDE.md
```

```
# Platforma rsk
```

```
Cinci repo-uri, un cluster K3s, ArgoCD.
```

```
| Folder | Ce e |  
|---|---|  
| `data-service/` | Spring Boot, consumer Kafka, scrie in MySQL |  
| `importer-service/` | Spring Boot, citeste MySQL, scrie pe product-topic |  
| `client-microserv-vite/` | frontend React + Vite, auth prin Keycloak |  
| `argo-ms-gitops/` | repo-ul de GitOps **citit de ArgoCD** |  
| `ms-gitops/` | repo-ul de GitOps **in care scrie CI-ul** |
```

```
Ultimele doua randuri nu sunt o greseala de scriere. Vezi /lant.
```

```
Namespace-uri: business, databases, messaging, monitoring.
```

```
Realm Keycloak: rsk. Domeniu public: icode.mywire.org.
```

```
Imaginile se publica pe Docker Hub sub ion21/.
```

```
Tag de image = SHA-ul scurt al commit-ului din repo-ul serviciului.
```

```
## Reguli
```

- Clusterul se citeste, nu se modifica. Orice schimbare pleaca dintr-un commit.
- Nu propune `kubectl apply`. Propune modificarea in repo-ul de GitOps.
- Cand raportezi o valoare dintr-un fisier, scrie si calea fisierului.

Poarta — o întrebare, două porniri

Repornește în `ms/` și întreabă, fără niciun alt context:

```
In ce namespace ruleaza data-service si de unde isi ia baza de date?
```

Gata când: răspunde `business` și MySQL-ul din cluster. Dacă bâjbâie, ai pornit în alt folder — verifică cu `pwd`.

Ce ai făcut de fapt: nu l-ai făcut mai deștept. l-ai dat harta pe care tu o ai în cap. Tot ce urmează în ghid se bazează pe harta asta — comanda, skill-ul și subagentul o moștenesc, nu și-o rescriu.

Când NU: `CLAUDE.md` nu e loc de documentație. Fiecare rând din el se citește la fiecare pornire, la fiecare conversație. Un fișier de trei pagini îți mănâncă din context exact spațiul de care ai nevoie ca să-ți citească repo-urile. Ține-l sub o pagină, și scoate din el orice poate fi aflat citind codul.

Piesa 3 — Comanda `/nume`: cererea salvată

Ce e: un fișier text care devine o comandă. Tot ce scrii în el ajunge în fața agentului atunci când tastezi `/nume`. E cererea pe care o repeți identic, pusă într-un loc din care o poate rula și un coleg.

```
ms/.claude/commands/lant.md
```

```
---
description: Compara tagul citit de ArgoCD cu tagul scris de CI
argument-hint: [nume-serviciu, optional]
allowed-tools: Read, Grep, Glob, Bash(git log:*), Bash(git show:*)
---
```

Verifica lantul GitOps pentru serviciile platformei. Daca primești un argument, verifica doar serviciul \$ARGUMENTS; altfel, toate trei.

Pasi

1. Pentru fiecare serviciu, gaseste tagul de imagine din repo-ul de GitOps pe care il citește ArgoCD (fișierele din `argo-ms-gitops/`).
2. Gaseste tagul scris de ultima rulare de CI (workflow-urile din repo-ul serviciului + ultimul commit al lui ci-bot in repo-ul tinta).
3. Compara-le.

Ce scoti

Un tabel cu: serviciu | tag citit de Argo | ****fișierul din care l-ai citit**** | tag scris de CI | ****fișierul in care a scris**** | OK sau DRIFT.

Daca un serviciu nu e deployat prin ArgoCD deloc, scrie asta explicit pe un rand al lui. Nu-l sari.

Nu raporta niciodata doar "OK": fara caile fișierelor, raportul nu e verificabil.

Nu modifica niciun fișier.

Trei lucruri de reținut din exemplul ăsta, pentru că sunt tot ce-ți trebuie ca să scrii orice altă comandă:

- `$ARGUMENTS` — se înlocuiește cu ce scrii după numele comenzii: `/lant importer-service`. Pentru argumente separate ai și `$1`, `$2`.
- `allowed-tools` — comanda rulează cu drepturile astea, nu cu toate. O comandă de verificare care are voie doar să citească nu poate strica nimic nici dacă textul ei e prost scris.
- `description` — ce vezi când tastezi `/` și se deschide lista.

Poarta — o unealtă care nu poate spune „DRIFT” nu e o unealtă

Asta e exercițiul pe care un om bun îl consideră evident și îl sare. Nu-l sări.

- Într-un repo, schimbă cu mâna o cifră din tag-ul din `values.yaml`. Nu comiți.
- Rulezi `/lant` din nou.
- **Trebuie** să iasă DRIFT, pe serviciul ăla, cu ambele tag-uri și ambele fișiere afișate.
- Pui cifra la loc. `git diff` gol.

De ce: dacă unealta ți-a spus „OK peste tot”, n-ai aflat că platforma e sănătoasă. Ai aflat că unealta a scris „OK”. Sunt două lucruri diferite și se despart exact atunci când te bazezi pe ea. E același reflex pe care îl ai când scrii un test: **un test care trece pe cod stricat nu e un test.**

Piesa 4 — Skill-ul: procedura pe care agentul o alege singur

Un skill e o procedură scrisă într-un folder, pe care agentul o deschide singur când situația i se pare potrivită. Nu o chemi pe nume. Nu apare în nicio listă pe care s-o răsfoiești în timp ce lucrezi. Agentul citește linia `description` a fiecărui skill pe care îl ai, o compară cu ce i-ai cerut tu, și decide.

De aici vine toată diferența practică: o comandă te ajută când îți amintești de ea. Un skill te ajută **când ai uitat de ea** — și asta e exact situația în care ai nevoie de ajutor. Procedura de verificat secretele înainte de push nu e utilă când ești atent; e utilă vineri la șapte seara.

Al doilea avantaj, care se vede abia la procedurile lungi: skill-ul e un **folder**, nu un fișier. `SKILL.md` stă în el, dar lângă el poți pune orice — un script, un șablon, o listă de tipare. Textul din `SKILL.md` e scurt și trimite la restul, care se citește doar la nevoie.

```
ms/.claude/skills/secrete/SKILL.md
```

```
---
name: secrete
description: Cauta secrete in clar (chei, parole, client secret) in
  repo-urile platformei. Se foloseste inainte de orice push sau commit,
  cand se adauga un fisier de configurare, si cand intreb daca e ceva
  sensibil in repo.
---

Cauti valori sensibile commise in clar. Toate cele cinci repo-uri sunt
publice pe GitHub, deci orice gasesti e deja public.

## Unde cauti
- `.env`, `*.env`, `public/config/*.json`, `src/**/vite-env.d.ts`
- manifeste: `values*.yaml`, `realm*.yaml`, `*-config.yaml`
- workflow-uri `.github/workflows/*.yaml` (valori scrise direct)

## Ce NU raportezi
- fisiere `SealedSecret` — sunt criptate, e normal sa contina siruri lungi
- `secrets.NUME` din workflow-uri — aia e forma corecta
- valori de test evident false (`changeme`, `test123`)

## Ce scoti
Tabel: fisier:linie | ce pare a fi | cat de expus (public? servit pe web?).
La final, un singur rand: care e primul de rotit si de ce.

## Ce nu faci
Nu stergi si nu rescrii nimic. O cheie stearsa dintr-un fisier ramane
in istoricul git — fixul e rotirea ei, nu stergerea. Propui, nu aplici.
```

Poarta — skill-ul nu se testează cerându-i-l pe nume

Dacă îi spui „folosește skill-ul secrete”, ai testat cititul, nu declanșarea. Testul adevărat are doi pași:

1. Declanșarea. Pornește o sesiune nouă și scrie o cerere **care nu conține cuvântul „skill”**:

```
Vreau sa fac push la client-microserv-vite. E ceva sensibil in el?
```

Trebuie să vezi că a deschis skill-ul, singur. Dacă nu, problema e aproape sigur în `description`: e scrisă pentru tine, nu pentru el. **Descrierea nu spune ce face skill-ul, spune când să fie folosit** — cu cuvintele pe care le folosești tu când ceri.

2. Că găsește ceva. Pune cu mâna, într-un fișier de configurare, o linie falsă de tipul `API_KEY=nu-e-reala-123`. Rulezi din nou. Dacă n-o raportează, skill-ul se uită în altă parte decât crezi. Ștergi linia.

Gata când: ambii pași trec. Abia atunci ai o procedură pe care te poți baza fără s-o supraveghezi.

Când NU: pentru ceva ce vrei să se întâmple **de fiecare dată, garantat**, skill-ul e unealta greșită — agentul decide dacă îl folosește, deci uneori nu-l folosește. Garanția e hook-ul. Skill-ul e pentru „de obicei, aici, așa se procedează”.

Piesa 5 — Subagentul: al doilea rol, cu contextul lui

Ce e: un al doilea agent, pornit de primul, cu instrucțiunile lui, drepturile lui și — partea importantă — **contextul lui separat**. Săpă cât trebuie și îți întoarce doar concluzia. Conversația ta rămâne curată.

```
ms/.claude/agents/sanatate-cluster.md
```

```
---
name: sanatate-cluster
description: Diagnostic read-only pe clusterul K3s si pe ArgoCD. Se
  foloseste cand ceva e picat (pod in CrashLoop, aplicatie OutOfSync,
  404 sau 502 pe un ingress) si vreau cauza, nu o reparatie.
tools: Bash, Read, Grep, Glob
---
```

Esti inginer de diagnostic pe platforma rsk. Regula fara exceptii:
READ-ONLY.

Ai voie
kubectl get / describe / logs / events / top; argocd app get / diff / list;
citit fisiere din cele cinci repo-uri; git log / diff / show.

Nu ai voie
Nicio comanda care schimba ceva: apply, delete, edit, scale, patch, rollout,
helm install/upgrade, argocd app sync, git commit/push.
Daca fixul cere o schimbare, o DESCRII: ce fisier din repo-ul de GitOps se
modifica si cu ce continut exact. Nu o aplici.

Ce intorci
Trei sectiuni, in ordinea asta, si nimic altceva:
SIMPTOM (ce se vede) -> CAUZA (mecanismul, nu suprafata) -> FIX PRIN REPO.
Daca nu poti ajunge la cauza, spune unde te-ai oprit si ce ti-ar trebui.
Nu ghici.

Exemplul pe care îl ai la îndemână chiar acum

Containerul de UI se construiește verde și servește 404. Într-o conversație normală, asta înseamnă zece `kubectl -uri` și trei fișiere citite, toate rămase în contextul tău. Deleget, arată așa:

```
Foloseste sanatate-cluster: de ce serveste 404 imaginea de UI,
desi build-ul a iesit verde?
```

Ce primești înapoi: simptom, cauză, fix prin repo — trei paragrafe. Pașii prin care a ajuns acolo au rămas în contextul lui, nu în al tău.

Poarta: cere-i, în aceeași sesiune, să repare ce a găsit. Trebuie să refuze și să-ți descrie modificarea în loc s-o aplice. Un subagent „read-only” care acceptă să scrie e un subagent read-only doar în titlu — și l-ai fi chemat exact când clusterul e deja stricat.

Când NU: pentru o întrebare de două fișiere e mai lent și mai prost — pornește fără contextul conversației tale, deci trebuie să-i explici de la zero. Subagentul își merită costul la treburi lungi, la treburi pe care vrei să le rulezi în paralel, sau când vrei **alte drepturi** decât ale sesiunii tale.

Piesa 6 — Hook-ul: ce se întâmplă fără să ceri

Ce e: o comandă de shell pe care o rulează Claude Code **la un eveniment**, nu la cererea ta. Singura piesă din ghid care nu depinde de decizia agentului: hook-ul rulează chiar dacă agentul „a uitat”, chiar dacă tu ai uitat, chiar dacă

cererea era formulată altfel.

Evenimentul	Când se declanșează	Ce faci cu el la tine
PreToolUse	înainte de o unealtă; o poate bloca	oprești un commit cu markeri de conflict sau cu secrete
PostToolUse	după ce unealta a rulat	rulezi <code>helm template</code> după orice modificare de chart
UserPromptSubmit	când trimiți un mesaj	adaugi automat contextul: branch-ul curent, contextul kubectl
Stop	când agentul a terminat	îți amintește ce a rămas necomis în working tree

ms/.claude/settings.json – se adaugă lângă "permissions"

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/pre-commit.sh"
          }
        ]
      }
    ]
  }
}
```

ms/.claude/hooks/pre-commit.sh – chmod +x

```
#!/bin/bash
cmd=$(jq -r '.tool_input.command // ""')

case "$cmd" in
  *"git commit"*) ;;
  *) exit 0 ;;
esac

if git diff --cached -U0 | grep -qE '\^(<<<<<<|=====|>>>>>>)'; then
  echo "BLOCAT: markeri de conflict in ce urmeaza sa comiti." >&2
  exit 2
fi

pat='^\+.*(secret|password|api_key)=[A-Za-z0-9/+]{{16,}}'
if git diff --cached -U0 | grep -qiE "$pat"; then
  echo "BLOCAT: pare un secret in clar in commit." >&2
  exit 2
fi

exit 0
```

Cum funcționează, în trei rânduri: hook-ul primește pe intrarea standard un JSON cu unealta și argumentele ei — de aici `jq -r '.tool_input.command'`. Ieșirea **codul 2** e singura care contează: blochează unealta, iar ce ai scris pe `stderr` ajunge înapoi la agent, care află de ce a fost oprit. Orice alt cod de ieșire lasă comanda să treacă.

Poarta — hook-ul se testează stricând, nu citind

Nu e un exemplu inventat: ai deja în istoric un commit cu markeri de conflict rămași în `Dockerfile`, comis fără să fie deschis. Hook-ul ăsta l-ar fi oprit. Deci:

- Scrie cu mâna `<<<<<< HEAD` pe primul rând al unui fișier oarecare, `git add`, și cere-i agentului să comită.
- Trebuie să fie blocat, și trebuie să vezi motivul în răspunsul lui.
- Scoți linia. Verifici cu `git diff --cached` că nu rămâne nimic.

De reținut, și e propoziția care justifică toată piesa asta: o regulă pe care ți-o amintești nu e o regulă. E o intenție. Regula e ce se întâmplă și când nu te gândești la ea.

Când NU: un hook prost e mai rău decât niciun hook — se declanșează la fiecare comandă, deci dacă e lent sau dacă blochează din greșeală, îți otrăvește toată ziua de lucru. Scrie-l strâmt: verifică întâi dacă evenimentul te privește (aici: dacă e chiar un `git commit`) și ieși repede cu 0 dacă nu.

Piesa 7 — MCP: unelte din altă parte

Ce e: un mod prin care Claude Code primește unelte de la un server din exterior — Jira, Sentry, o bază de date, un serviciu intern. Se declară într-un fișier din rădăcina proiectului sau se adaugă cu `claude mcp add`, iar uneltele apar în sesiune ca `mcp__<server>__<unealta>`.

```
ms/.mcp.json - forma fișierului
{
  "mcpServers": {
    "nume-server": {
      "command": "npx",
      "args": ["-y", "pachetul-serverului"],
      "env": { "TOKEN": "${TOKEN_DIN_MEDIU}" }
    }
  }
}
```

Două lucruri înainte să adaugi vreunul. Întâi: un server MCP e cod străin care rulează pe mașina ta, cu token-urile tale — se aplică aceeași judecată ca la orice dependență nouă, plus faptul că agentul îi va cita răspunsurile ca pe niște instrucțiuni. Al doilea: permisiunile funcționează și aici, sub forma `mcp__nume-server__nume-unealta`, deci un server nou se pune cu aceleași `deny`-uri ca restul.

Când NU — și la tine ăsta e răspunsul, deocamdată: nu-ți trebuie. Tot ce ți-ar da un server MCP ai deja prin linia de comandă: `kubectl`, `argocd`, `helm`, `gh`, `docker`. Agentul le rulează direct, sub regulile tale de permisiuni, fără nimic nou de instalat și fără un token în plus care să iasă din casă. MCP începe să merite când ai o unealtă **fără CLI**. Verifici ce ai pornit cu `/mcp`.

Cele patru piese, una lângă alta

Aceeași întrebare pusă pe fiecare, ca să se vadă unde sunt diferite. Întrebarea: „*verifică lanțul GitOps*”.

Piesa	Se declanșează	Context	Alegi asta când...
CLAUDE.md	totdeauna, la pornire	al tău	...e ceva ce trebuie să știe înainte de orice întrebare
comandă /lant	doar dacă o scrii tu	al tău	...o ceri des și vrei exact același lucru de fiecare dată

Piesa	Se declanșează	Context	Alegi asta când...
skill <code>secrete</code>	agentul decide, după <code>description</code>	al tău	...vrei să se aplice și când tu ai uitat de ea
subagent	delegat, explicit sau nu	separat	...e o săpătură lungă, sau vrei alte drepturi decât ale tale
hook	pe eveniment, garantat	niciunul (e shell)	...trebuie să se întâmple de fiecare dată , fără excepție

Regula de alegere, într-un rând: dacă poți trăi cu „de obicei”, scrii skill. Dacă ai nevoie de „întotdeauna”, scrii hook. Dacă vrei să decizi tu de fiecare dată, scrii comandă.

Când nu se declanșează — tabelul de depanare

Simptom	Cauza, în ordinea probabilității	Verifici cu
comanda <code>/lânt</code> nu apare în listă	fișierul nu e în <code>.claude/commands/</code> al folderului din care ai pornit; sau ai pornit în alt folder	<code>pwd</code> , apoi <code>ls .claude/commands/</code>
skill-ul nu se declanșează niciodată	<code>description</code> spune ce face skill-ul, nu când să fie folosit; nu conține cuvintele tale	rescrii descrierea cu fraza exactă pe care ai folosit-o când ai cerut
<code>deny</code> nu prinde comanda	regula se potrivește pe forma comenzii; agentul a chemat-o altfel, sau dintr-un script	<code>/permissions</code> + îi ceri să-ți arate comanda exactă
hook-ul nu rulează	fișierul nu e executabil; <code>matcher</code> nu se potrivește cu numele uneltei; lipsește <code>jq</code>	<code>/hooks</code> , apoi rulezi scriptul cu mâna și îi dai JSON pe intrare
răspunsurile ratează detalii	contextul e plin — conversație lungă sau <code>CLAUDE.md</code> umflat	<code>/context</code> , apoi <code>/clear</code>
subagentul întreabă lucruri pe care le știi	pornește fără contextul conversației tale; e normal	pui în cerere ce-i trebuie, sau nu delegi deloc

Ordinea în care le adaugi

Nu toate deodată. Ordinea de mai jos nu e didactică, e ordinea în care fiecare piesă devine **necesară** din cauza celei dinainte:

#	Piesa	De ce acum
1	permisiuni	înainte de prima unealtă, nu după prima pățanie
2	<code>CLAUDE.md</code>	fără harta platformei, orice altă piesă lucrează pe presupuneri
3	o comandă	prima verificare pe care o repeți — la tine, <code>/lânt</code>
4	un skill	prima procedură pe care ai uitat-o deja o dată
5	un hook	prima regulă pe care ai încălcat-o fără să vrei
6	un subagent	prima săpătură care ți-a umplut contextul degeaba

Punctele 4, 5 și 6 au același declanșator: o pățanie a ta. Nu construi piesa înainte să apară problema pe care o rezolvă — o să scrii un skill pentru munca despre care presupui că o faci, nu pentru cea pe care o faci într-adevăr. De asta jurnalul de o săptămână din lecție e mai important decât pare: din el ies uneltele, în ordinea corectă.

Ce duci acasă

Un singur lucru, și nu e o unealtă. E întrebarea cu care citești orice iese din atelierul ăsta:

De unde știu că unealta asta poate să-mi spună „nu”?

O unealtă care n-a spus niciodată DRIFT, un skill care n-a găsit niciodată un secret, un hook care n-a blocat niciodată un commit — toate trei arată identic cu „totul e în regulă” și identic cu „nu funcționează”. Se despart într-un singur fel: le strici tu ceva, cu mâna, și verifici că se plâng. Cinci minute pentru fiecare piesă, o singură dată.